GUIDA PRATICA / PRACTICAL GUIDE

Gazebo on Parallels

Step-by-step guide to the global configuration adopted on the VM

Ubuntu 24.04 ARM64 | ROS 2 Jazzy | Gazebo Harmonic

Revisione 4 / Revision 4 - 29 October 2026

Primary procedure	Purpose
Private Mesa + modified OGRE	Rendering and sensors in the tested case
gz / gzserver delegates + defaults	Activation through ordinary commands
GUI scene refresh	LiDAR visual at the sensor
Backups and rollback	Restoring original entry points

The global profile and the PR are separate results. This guide retains the profile procedure and adds the later evidence.

Approximately 60 GUI camera updates/s observed. A second clean VM remains to be tested.

A step-by-step guide from Ubuntu 24.04 ARM64 with ROS 2 Jazzy to Gazebo Harmonic with working camera and LiDAR sensors. Revision 4, October 29, 2026: the VM-wide configuration.

This procedure reproduces the workaround verified on a Mac M5 Max, Parallels Desktop/Tools 27.0.3, Ubuntu 24.04 ARM64, Gazebo Sim 8.15.0 and gz-rendering 8.2.3. It is not an official Gazebo or Mesa patch, and it does not establish that every Parallels VM has the same defect.

Verified result	Measurement
Hardware-accelerated 3D GUI	Approximately 60 camera updates/s
TurtleBot3 LiDAR	360 rays, approximately 10 Hz, correct distances
Full HD RGB camera	Correct images, approximately 8-11 images/s
Accuracy in the 2D test	Maximum error approximately 1.5 mm over 131 rays

The kit contains the local source files and scripts for the VM-wide configuration used on the measured VM. Everyday commands remain gz sim and ros2 launch: installed delegates activate the private profile automatically. No custom launcher needs to be typed. Coverage applies to the Jazzy entry points listed in step 8, not every executable or application embedding Gazebo libraries.

Important for the VM already corrected during the diagnosis: do not repeat the installation. This guide describes how to reconstruct the solution from a compatible standard installation; a complete run on a second clean VM is still pending. On the existing VM, you can use the verification sections.

- Run all installation commands inside the Ubuntu VM, except for the initial operations in the Parallels interface on the Mac.

- Run one command block at a time. Check the stated result before moving on to the next block.

- TurtleBot files, worlds and SDF/URDF models are not modified.

HOW TO USE THIS REVISION

This is the guide to the VM-WIDE configuration actually adopted on the measured system, separate from PR #1400. Steps 1-15 remain the primary procedure: private Mesa, modified OGRE component, entry-point delegates, defaults library and GUI scene refresh. Sections 16-19 add reasons, later evidence and alternatives without replacing this path with the overlay.

The folder contains gazebo_jazzy_parallels_kit.zip and its extracted kit/ directory. Transfer the ZIP into Ubuntu to follow step 4. The kit downloads Mesa and dependencies with apt; this is not an offline Ubuntu distribution. The original gz-rendering 8.2.3 archive is also retained at alternativa_overlay_8_2_3/overlay/downloads/gz-rendering-8.2.3.tar.gz. You may copy it to kit/build/downloads/ before the source phase, keeping its filename, to avoid downloading it again.

The approximately 60 figure refers to measured GUI camera updates on the recorded system. It does not guarantee 60 display-presented frames per second in every world or 60 Hz sensor output. Check image and scan correctness before discussing smoothness.

1. CHECK THE VM AND STARTING ENVIRONMENT

On the Mac: use an Ubuntu 24.04 ARM64 VM on Apple silicon. Before making changes, create a Parallels snapshot if your edition supports it, or back up the powered-off VM. Then start Ubuntu and sign in to the desktop.

The tested configuration had 8 vCPUs and 16 GB of RAM. These are the resources used in the test, not an established minimum requirement. Leave free disk space for source files, compilation and the runtime: allow at least a few GB.

Parallels uses VirGL/Virtio GPU for Linux ARM acceleration; its actual availability must be checked inside Ubuntu. Use Parallels Tools that match your Parallels version. The verified combination was 27.0.3 build 58680 [2].

Open a terminal on the Ubuntu desktop and check the system and architecture:

```
cat /etc/os-release
uname -m
dpkg --print-architecture
```

Expected: Ubuntu 24.04, aarch64 and arm64. If you have amd64, another distribution or another Ubuntu version, do not use this kit without adapting and testing it.

```
source /opt/ros/jazzy/setup.bash
echo "$ROS_DISTRO"
```

Expected: jazzy. If /opt/ros/jazzy/setup.bash does not exist, first complete the official Jazzy installation for Ubuntu 24.04 [1]. The following procedure assumes that Jazzy is already installed.

2. INSTALL GAZEBO AND THE REQUIRED TOOLS

Continue in the Ubuntu terminal. The recommended pairing for Jazzy is Gazebo Harmonic, available through vendor packages in the ROS repository. You do not need to add a second Gazebo repository for this procedure [3].

```
sudo apt update
sudo apt install ros-jazzy-ros-gz
sudo apt install build-essential cmake pkg-config curl ca-certificates \
  patch unzip python3 python3-pil mesa-utils libogre-1.9-dev \
  libgl-dev qtbase5-dev libprotobuf-dev protobuf-compiler
```

These commands install Gazebo and build dependencies. The workaround does not require downgrading system Mesa. Historical Mesa packages will only be downloaded and extracted later.

```
source /opt/ros/jazzy/setup.bash
command -v gz
gz sim --version
ros2 pkg prefix ros_gz_sim
```

The tested installation used Gazebo Sim 8.15.0 and ros_gz_sim 1.0.24. The gz command comes from the Jazzy gz_tools vendor package.

```
grep "^Version:" \
  /opt/ros/jazzy/opt/gz_rendering_vendor/lib/pkgconfig/gz-rendering8.pc
pkg-config --modversion OGRE
```

Expected: gz-rendering 8.2.3 and OGRE 1.9.0. The gz-rendering check reads the .pc file directly: sourcing ROS alone may not add vendor pkg-config directories to PKG_CONFIG_PATH.

If the versions differ, stop. Do not blindly replace version numbers, libraries or hash checks: the kit is built and installed only for the tested combination.

3. VERIFY THAT VIRGL IS ACTUALLY ACTIVE

Run this check from a terminal on the Ubuntu desktop, not from the macOS terminal:

```
glxinfo -B
```

Look for direct rendering: Yes, a renderer containing virgl and, when shown, Accelerated: yes. The verified system reported virgl (Apple M5 Max (Compat)). The GPU name will differ on other Macs.

If you see llvmpipe or softpipe, rendering is running on the CPU. In that case, do not proceed on the assumption that the kit can enable a virtual GPU that the VM does not expose.

```
printenv | grep -E \
  "^(LIBGL_|GALLIUM_|MESA_|LD_PRELOAD=|QT_QPA_PLATFORM=)"
```

A fresh installation should not have graphics overrides left over from previous attempts. If you have custom settings, check them first. For its own process, the Gazebo profile removes common overrides that force software rendering or GL versions; it does not rewrite the global shell configuration.

```
sudo dmesg | grep -i virgl
```

This is an additional check suggested by Parallels [2]. The decisive application-level check is still the actual renderer reported by glxinfo.

A high OpenGL version number does not guarantee that every Gazebo shader works. In the verified case, the driver reported OpenGL 4.0, but some OGRE 2 paths and sensor rendering passes remained problematic.

If DISPLAY is unavailable because you are connected over SSH, perform this first check in a graphical terminal inside the VM. The kit does not enable X11 forwarding and does not require rendering on the Mac over SSH.

4. COPY AND OPEN THE KIT INSIDE THE VM

Download gazebo_jazzy_parallels_kit.zip and copy it into the VM's Downloads directory. You can use a Parallels shared folder or your preferred file-transfer method. The file must be in the Ubuntu filesystem before you run these commands.

```
mkdir -p "$HOME/gazebo-parallels"
unzip "$HOME/Downloads/gazebo_jazzy_parallels_kit.zip" \
  -d "$HOME/gazebo-parallels"
cd "$HOME/gazebo-parallels/kit"
sha256sum -c SHA256SUMS
```

Expected: every file is reported as OK. The checksums verify the integrity of the kit contents; they are not a digital signature from the software vendor.

File or directory	Purpose
prepare.sh	Preparation in four phases
sources/	Patch, Ruby profile, entry points and defaults library
CMakeLists.txt	Reproducible component build
install.py	Verification and installation with backups/diversions
rollback.py	Restore the original entry points
tools/ and tests/	GPU, sensor, distance and GUI checks

Return to this directory at the start of each new session working with the kit. The commands in steps 5-9 assume this location.

```
source /opt/ros/jazzy/setup.bash
```

Do not run prepare.sh with sudo: it should prepare files in your user directory. Root privileges are required only for the explicit installation step.

5. PREPARE PRIVATE MESA WITHOUT A SYSTEM DOWNGRADE

Run only the Mesa phase:

```
bash prepare.sh mesa
```

The script checks Ubuntu 24.04 arm64, gz-rendering 8.2.3 and OGRE 1.9.0. It then uses apt-get download and dpkg-deb -x. It downloads packages but does not register them as installed.

Private package	Pinned version
libegl-mesa0, libgbm1	24.0.5-1ubuntu1
libgl1-mesa-dri, libglapi-mesa	24.0.5-1ubuntu1
libglx-mesa0	24.0.5-1ubuntu1
libllvm17t64	1:17.0.6-9ubuntu1
libxcb-dri2-0	1.15-1ubuntu2

The .deb files remain in build/downloads; their extracted contents are in build/mesa-root. LLVM and XCB dependencies are part of the verified combination: copying only virtio_gpu_dri.so is not sufficient.

```
dpkg-query -W -f='${Version}\n' libgl1-mesa-dri
```

Record the system version. On the verified VM, it remained 25.2.8-0ubuntu0.24.04.4 even after the workaround was activated. It may be different on another installation.

If apt cannot find a pinned version, first check that the official Ubuntu Noble sources are configured and that apt update succeeded. Do not replace the command with sudo apt install of an older Mesa, and do not choose arbitrary versions. The kit must stop if the required packages are unavailable.

6. DOWNLOAD THE SOURCE AND APPLY THE PATCH

```
bash prepare.sh source
```

This phase downloads the official gz-rendering8_8.2.3 tag, checks it against the SHA256 pinned in the kit, extracts the source into build/src and applies sources/gazebo-virgl.patch.

The patch makes three functional changes:

- It forces OGRE 1 light shadows off. The combined workaround made the observed GUI usable, but an isolated GUI A/B recording is still needed to establish the contribution of this change.

- For single-row LiDAR, it retains the original three-row internal workaround, center sampling and consistent aspect ratio. Public dimensions stay as requested. Later tests succeeded without padding in the controlled cases after the other corrections; this guide preserves the VM-wide profile actually adopted.

- It includes restoration of the LiDAR internal camera orientations from the upstream gz-rendering8 branch, associated with PR 1378 [4].

The 2D case also required avoiding a non-finite vertical coordinate produced by the general formula when the vertical field of view is zero. No artificial robot offset is introduced, and published distances are not altered.

```
less sources/gazebo-virgl.patch
```

You can read the patch before continuing. The correction is in the graphics component, not in TurtleBot files.

If the SHA256 check or patch application fails, stop. Do not use patch --force. If an earlier preparation was left incomplete, use a fresh copy of the kit instead of reusing source files in an uncertain state.

7. BUILD AND ASSEMBLE THE RUNTIME

```
bash prepare.sh build
bash prepare.sh assemble
```

The build uses two parallel jobs to limit the load on the VM. You do not need to run colcon over the entire ROS distribution, and you do not need to install the gz-rendering project into the system.

The assemble phase creates a self-contained runtime in build/runtime containing:

- the corrected OGRE 1 component and a link with the library name expected by Gazebo;

- private Mesa, the VirGL driver and dependencies;

- a profile that selects these libraries only in Gazebo processes;

- a defaults library for the native server and LiDAR GUI synchronization;

- source files, a hash manifest and licenses for the copied packages.

```
ls build/runtime
ls build/runtime/engine
ls build/runtime/mesa/dri
```

Expected: profile.rb, libgazebo_virgl_defaults.so, and the engine and mesa directories; libgz-rendering-ogre.so inside engine; and virtio_gpu_dri.so in the driver directory.

The profile explicitly selects OGRE 1/OpenGL, Qt xcb and private Mesa, and removes common software-rendering overrides. The component is loaded by absolute path so that the standard vendor copy is not selected first.

If assemble reports that build/runtime already exists, do not blindly overwrite an existing runtime. To repeat the procedure on a new machine, start from a fresh kit directory.

8. CHECK AND INSTALL THE DEFAULT CONFIGURATION

Close Gazebo and ROS launches before installation, saving any work. From the kit directory, first run the check that makes no changes:

```
python3 install.py --check
```

Expected: the runtime and entry points are verified; no changes are made. The script checks the tested original files and rejects an existing profile or diversion. If your VM has already been corrected, the STOP is expected: do not reinstall.

```
sudo python3 install.py --install
```

Installation creates the private runtime under /usr/local/lib/gazebo-virgl-defaults/runtime-v2. It preserves the original entry points with dpkg-divert and installs two Ruby delegates compatible with ROS launches:

```
/opt/ros/jazzy/opt/gz_tools_vendor/bin/gz
/opt/ros/jazzy/lib/ros_gz_sim/gzserver
```

The originals become gz.vendor and gzserver.vendor. Backups and the manifest are stored under /var/backups/gazebo-virgl-defaults and /usr/local/share/gazebo-virgl-defaults.

The entry-point hashes are pinned to the tested versions. If they differ, the script stops before making changes. This restricts the procedure to the verified combination; it does not mean that bypassing the check on other releases is safe.

System Mesa, /etc/ld.so.preload, .bashrc, SDF/URDF files, worlds and TurtleBot files are not modified. The runtime also applies to other models using the same Jazzy/Harmonic entry points, subject to testing the sensors and features actually required.

9. VERIFY THE GRAPHICS DEFAULTS

Open a new Ubuntu terminal. Check the renderer only within the Gazebo profile environment:

```
cd "$HOME/gazebo-parallels/kit"
bash tools/private_glxinfo.sh
```

Expected: VirGL, active acceleration and Mesa 24.0.5. Then run glxinfo without the profile: it should report the system Mesa version, which has not been downgraded.

```
glxinfo -B
```

Start a standard world using the normal command:

```
source /opt/ros/jazzy/setup.bash
gz sim -r shapes.sdf
```

Rotate the camera and observe the scene for at least one minute. Check colors, geometry, absence of flickering and response to camera controls. Initial Fuel model downloads may require network access and some time.

To view the GUI counter, add the Camera FPS panel from the plugin menu. Its value refers to the GUI camera; it is not the robot camera frame rate, nor an independent measurement of frames presented by the display.

Close this test before starting TurtleBot. Do not leave two non-isolated servers using the same world and topics: they can confuse clock, scan and image measurements.

10. TEST TURTLEBOT, CAMERA IMAGES AND SCANS

TurtleBot is an additional test, not a required dependency of the profile. On a new VM, install the model packages and an image viewer:

```
sudo apt install ros-jazzy-turtlebot3-gazebo ros-jazzy-rqt-image-view
source /opt/ros/jazzy/setup.bash
export TURTLEBOT3_MODEL=waffle
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

Do not add rendering options to the launch command. TURTLEBOT3_MODEL selects the model required by the TurtleBot package; it is not a workaround variable.

In a second terminal:

```
source /opt/ros/jazzy/setup.bash
ros2 topic list
ros2 run rqt_image_view rqt_image_view
```

Select /camera/image_raw in the image viewer. You should see the environment, not just a window or an existing topic. Check /scan with:

```
ros2 topic hz /scan
```

Expected: approximately 10 Hz. To inspect the contents, ranges and image in a results directory:

```
cd "$HOME/gazebo-parallels/kit"
mkdir -p risultati
cd risultati
python3 ../tools/check_sensors.py
```

After approximately 23 seconds, sensor-validation.json and turtlebot-camera.png are saved. In the TurtleBot world, returns must vary; all rays at 0.12 m mean the defect has not been resolved. Conversely, an entirely out-of-range scan may be correct in an empty world: the criterion depends on the scene.

11. CHECK DISTANCES AGAINST A KNOWN WALL

Close TurtleBot. The kit contains a scene with a wall 2 meters in front of a static sensor. Use a dedicated partition to separate the test from other Gazebo processes.

Terminal A, from the kit directory:

```
source /opt/ros/jazzy/setup.bash
export GZ_PARTITION=gazebo_kit_check
gz sim -r -s "$HOME/gazebo-parallels/kit/tests/known-distance.sdf"
```

Terminal B:

```
source /opt/ros/jazzy/setup.bash
export GZ_PARTITION=gazebo_kit_check
cd "$HOME/gazebo-parallels/kit"
python3 tools/validate_known.py
```

Expected: passed: true, a front distance of approximately 2 meters and a maximum error below 0.02 m, the acceptance tolerance for this test. The measured maximum over the 131 checked rays was approximately 0.00147 m. The JSON result is saved in the current directory.

Stop terminal A with Ctrl+C. For the 3D test, start this command instead:

```
gz sim -r -s \
  "$HOME/gazebo-parallels/kit/tests/known-distance-3d.sdf"
```

In terminal B, with the same partition still set:

```
python3 tools/validate_known.py --3d
```

Expected: 1083 rays and passed: true. After the test, close the server and test terminals, or run unset GZ_PARTITION before returning to everyday commands.

12. CHECK THE LIDAR VISUAL POSITION

In the TurtleBot world, add the Visualize Lidar panel, refresh the topic list and select /scan. The rays should originate at the sensor on the robot and follow its movement.

An additional GUI problem was observed: the panel could not find the entity associated with the topic and drew the scan near the world origin. This is separate from distance computation.

Revision 3 of the kit library automatically requests a complete scene state from GuiRunner when you select the topic. It does not change robot coordinates, ROS frames or published measurements.

Correction test: the test model was placed at (-2, 1, 0.5), and the LiDAR visual was measured at exactly (-2, 1, 0.5). A separate test with the sensor also rotated confirmed pose-consistent distances, with less than 4 mm error over the checked rays.

Alternative manual recovery, if you are using a GUI session opened before the update or need to investigate the problem:

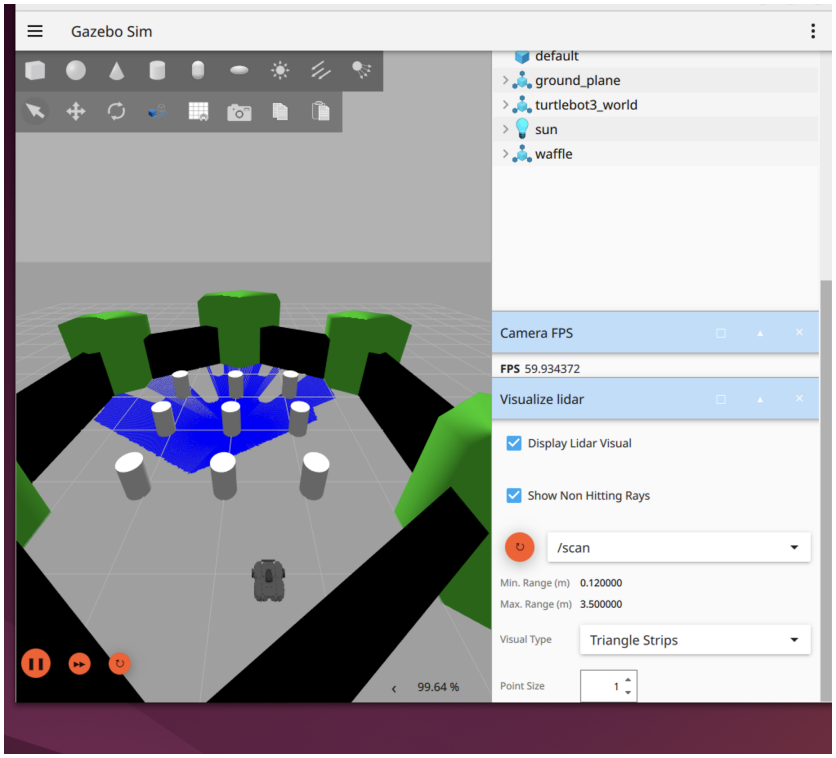
```
source /opt/ros/jazzy/setup.bash
cd "$HOME/gazebo-parallels/kit"
./build/engine/refresh_gui_state default
```

Replace default with the actual world name if it differs. With no argument, the utility requests the states of the available worlds. The GUI must already be connected. This command synchronizes graphics state; it does not move the robot in the physics engine.

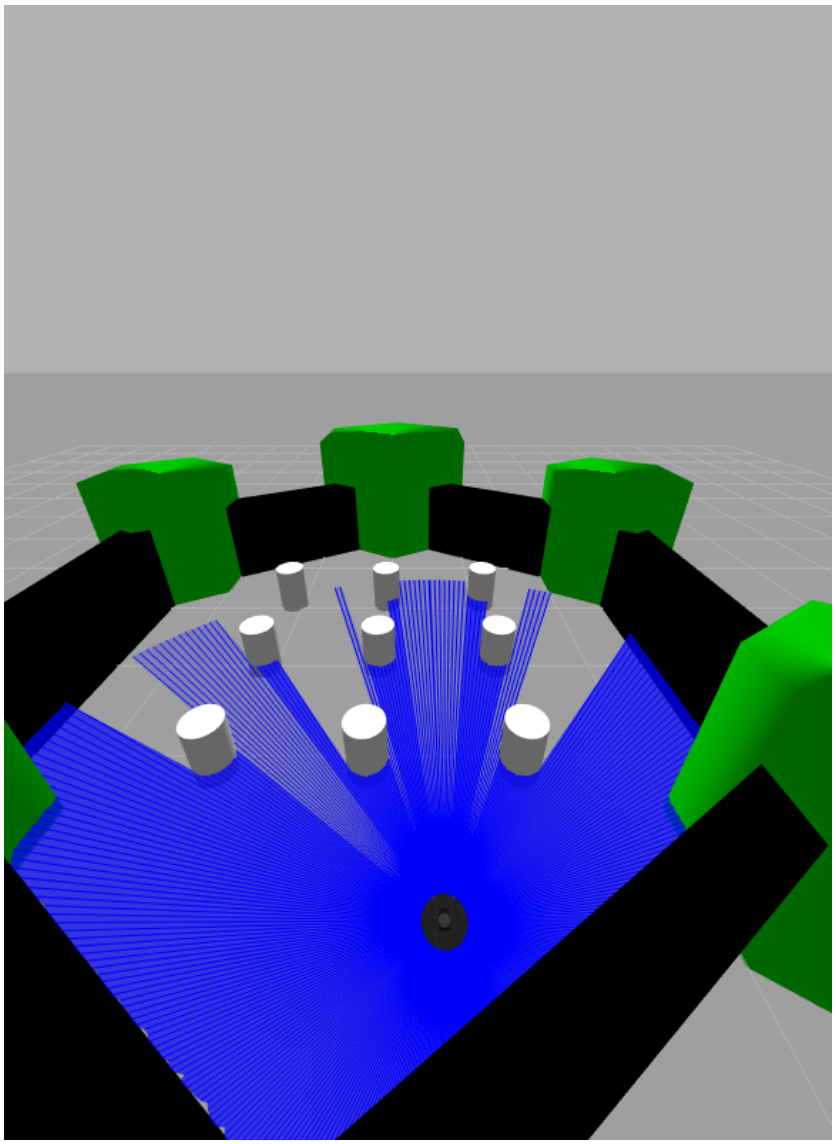
Do not compensate for this defect by adding meters to the LiDAR pose in the model file or changing frames: doing so would introduce an error into data that was already correct.

THE OBSERVED CASE: BEFORE AND AFTER

Prima / Before



Dopo / After



Before: the robot is near the bottom of the scene, while the blue scan is near the world center. The Visualize Lidar log reported that it could not find the sensor entity.

After retrieving the complete state: the blue scan originates at the robot. The two images use different viewpoints; the verification criterion is whether the rays originate at the sensor, not whether individual pixels match.

To distinguish visualization problems from measurement problems, looking at rays is not enough: you also need a scene with known distances and a test with a translated or rotated sensor. These checks were performed during the diagnosis.

13. REALISTIC PERFORMANCE AND USE WITH OTHER MODELS

Check on the case-study VM	Result
GUI, 85 five-second windows	Median 59.95; minimum 55.36 updates/s
1920x1080 RGB camera	Approximately 8-11 images/s over ROS
TurtleBot3 LiDAR	360 rays, approximately 10 Hz
2D wall, 131 checked rays	Maximum error approximately 1.5 mm
3D wall, 183 checked rays	Maximum error approximately 0.95 mm
Robot movement	Images and scans change consistently

The camera's nominal 30 Hz rate at Full HD was not reached. Do not confuse it with the approximately 60 GUI updates per second. Disabling antialiasing did not improve this case and is not part of the adopted configuration.

For a new application, assess resolution, sensor count, update rates, world complexity and algorithm requirements separately. A lower resolution may be a project choice, but the kit does not impose one on existing models.

The changes are in the runtime and Gazebo entry points, so they do not depend on TurtleBot. RGB camera output, horizontal 2D LiDAR, one 3D LiDAR case, ROS launch and the native server were verified. Depth/RGB-D, segmentation, thermal sensors, advanced materials and other cases were not tested by this procedure.

Light shadows remain disabled. If an experiment specifically requires those capabilities or OGRE 2-specific features, this workaround must be assessed against the experiment's requirements.

14. EVERYDAY USE, UPDATES AND ROLLBACK

Before changing worlds, stop the previous launch with Ctrl+C in its terminal and wait for the processes to exit. In ROS launches, the GUI and server may be separate: closing only the window does not guarantee that the server has stopped. A new GUI can connect to the old server and still show TurtleBot, even when another world was requested.

After installation, use the normal entry points:

```
source /opt/ros/jazzy/setup.bash
gz sim -r shapes.sdf
```

Alternatively, use the ROS launch for your selected model, such as the TurtleBot launch in step 10. You do not need to keep the kit directory open or manually export LD_PRELOAD and Mesa variables. The shell still needs the normal ROS environment.

To check that the TurtleBot package has not been modified:

```
dpkg -V ros-jazzy-turtlebot3-gazebo
```

On the case-study VM, this command reported no differences. For other models, check the corresponding package or model repository.

After major ROS, Gazebo, Parallels or Mesa updates, repeat the graphics and sensor checks. The private runtime is not automatically updated by apt. Do not apply this kit to a different Gazebo major version.

To roll back, first close the launch processes and Gazebo, then run:

```
sudo /usr/local/sbin/gazebo-virgl-defaults-rollback
```

The script removes the managed delegates and restores the .vendor entry points using dpkg-divert. It does not reinstall Mesa. The runtime stays on disk but is no longer loaded through the restored entry points. Open a new terminal and restart Gazebo.

GZ_VIRGL_DEFAULTS=0 provides a single-process opt-out for diagnostics; it is not a tested alternative solution. Full rollback was reviewed but was not executed on the working VM, to avoid undoing the requested configuration.

15. IF A STEP FAILS

Symptom	Check and action
Version or hash mismatch	Stop: the kit does not cover that combination. Do not remove the check.
Missing SDK or headers	Check step 2, the Jazzy environment and the first CMake error. Do not copy libraries from other operating systems.
Historical Mesa package unavailable	Check the official Noble repositories and apt update. Do not downgrade the system.
Cannot open the display	Try an Ubuntu graphical terminal. Over SSH, check DISPLAY and session authorization.
llvmpipe renderer	Check VirGL in the VM and compare normal and private glxinfo output.
Black camera image or minimum LiDAR ranges	Check that the process loads the private runtime; topic existence alone is not enough.
Rays far from the robot	Check topic selection and GUI synchronization in step 12; do not move the model.
The old world always opens	Stop the previous launch with Ctrl+C and check for servers left running in the background. Closing the GUI is not enough.
Unexpected rates or clock behavior	Check for two servers on the same world/topics. Close your test processes.

To inspect current processes without terminating them:

```
ps -u "$USER" -o pid,args | grep "gz sim"
```

A process started through the profile should internally show --render-engine followed by the private component path. The command typed by the user remains the normal one.

To capture a log from a new test:

```
gz sim -v 4 -r shapes.sdf 2>&1 | tee gazebo-prova.log
```

Keep logs, version information, renderer details and test results. You do not need SSH passwords or other secrets to share a reproducible diagnosis.

16. WHY ORDINARY COMMANDS ACTIVATE THE PROFILE

The VM-wide configuration covers two specific entry points. The gz file in gz_tools_vendor remains Ruby because ros_gz_sim can invoke it as ruby <path-to-gz> sim ...; a shell replacement would break that contract. Its delegate preserves the world arguments and executes the original gz.vendor. Only the sim subcommand, excluding help/version and opt-out, loads profile.rb and inserts the absolute OGRE plugin path and OpenGL backend. It does not hardcode a TurtleBot world.

The profile selects private Mesa through LD_LIBRARY_PATH and LIBGL_DRIVERS_PATH for the process and its children. It sets Qt xcb and adds libgazebo_virgl_defaults.so to LD_PRELOAD. It does not write /etc/ld.so.preload; other desktop applications retain system Mesa. runtime-v2 names the directory layout; the installation manifest has revision 3, including the LiDAR GUI refresh.

The gzserver delegate covers the native ros_gz_sim server, which need not use the gz CLI. It loads the same environment and executes gzserver.vendor. The defaults library intercepts the C1/C2 ServerConfig constructors, calls the real constructor first and sets GUI/server engine and API defaults. C1/C2 are separate C++ ABI entry points, not user settings. The library does not render shaders or enable a virtual GPU by itself.

For the CLI path, profile.rb removes the engine/backend options listed in its RENDER_OPTIONS and inserts its own. This forces the profile selection; it does not merely suggest defaults while preserving every explicit engine option. For the native server it sets the initial ServerConfig values; an application overriding those values later needs a separate test. Alternative binaries, containers, other ROS installations and applications embedding libgz-sim are outside a blanket coverage claim.

The second task is independent: after VisualizeLidar::OnTopic, the library calls the real handler, locates GuiRunner and queues RequestState through Qt. This refreshes GUI scene state. It does not repair incorrect range measurements, TF or the physical sensor pose.

Inspect the diversions and manifest without changing them:

```
dpkg-divert --list | grep gazebo
cat /usr/local/share/gazebo-virgl-defaults/manifest.json
ls -l /opt/ros/jazzy/opt/gz_tools_vendor/bin/gz*
ls -l /opt/ros/jazzy/lib/ros_gz_sim/gzserver*
```

Not every diversion line contains the word gazebo; also check both entry paths shown by ls. To verify loaded libraries, first identify the actual server or GUI PID, not an already exited Ruby delegate:

```
ps -u "$USER" -o pid,ppid,args | grep -E 'gz sim|gzserver|gz-sim'
```

Replace 12345 with that real PID and inspect its mappings:

```
PID=12345
cat "/proc/$PID/maps" | grep -E 'gazebo-virgl|GLX_mesa|virtio_gpu|libgallium'
```

Expect the private runtime and matching graphics libraries. If absent, check the selected process and logs. Installed files alone do not prove that a running process loaded them. Do not publish its entire environment, which may contain private data.

17. UPDATED REASONS AND LIMITS OF EACH INTERVENTION

Private Mesa 24.0.5. This is the combination that produced valid images in the recorded controls, not a demonstrated optimum among all Mesa releases. In the ablation, the noisy RGB camera returned black frames with Mesa 25.2.8/VirGL even with shadows off. Mesa 24/VirGL and llvmpipe 25 returned valid images. Disabling shadows alone therefore does not explain that camera failure. Mesa 24.2 was not tested, so there is no basis to reject or prefer it. The private libraries remain frozen until deliberately updated.

Disabled shadows. The profile patch makes lights ignore cast_shadows. This sacrifices a feature as part of the workaround; it is not a general shadow-map repair. The combined result is documented. A GUI recording changing this single intervention while holding everything else constant remains pending. Image publication tests do not measure display flicker.

Three internal LiDAR rows. Three was a practical choice because the middle row center is v=0.5. It does not establish that VirGL cannot support one-row textures. The later 53-trial ablation also succeeded with two rows and without padding in the controlled cases after the other corrections. The installed profile retains its historical padding; it has not silently been replaced by the PR patch.

Upstream corrections. Pixel centering (#1370) and camera orientation restoration (#1378) are separate upstream contributions credited to their authors. The UV guard instead avoids a vertical 0/0 expression for one output row. Later PR tests added the single-column guard and camera selection from the actual resampled row count. Not all these later additions are present in the original global kit.

Delegates, preload and dpkg-divert. These make the profile automatic for the ordinary commands requested by the owner. They are unnecessary for reproducing the numerical gz-rendering defect. They depend on Jazzy/Harmonic paths and ABI symbols, which updates can break. Backups, a manifest, hash gates and rollback are integral parts of this method.

Wrong world opening. The profile does not force TurtleBot. During the investigation, a retained test server caused a new GUI to connect to the previous world. Stop the launch and inspect its processes before changing worlds. Terminate only PIDs identified as belonging to your test.

18. SEPARATE ALTERNATIVES AND CONTRIBUTIONS

alternativa_overlay_8_2_3/ contains the earlier colcon overlay. Five short functional tests passed on the existing system: narrow LiDAR, full-circle LiDAR, three rows, a translated/rotated scene and plain/noisy cameras. It respects normal SDF shadows and installs neither VM-wide delegates nor the GUI refresh. Sourcing this overlay alone does not make OGRE 1 the default for every ROS launch. It helps isolate rendering and compare drivers but does not replace the automatic configuration documented here.

For its commands, consult chapters 8-9 of report 03, using alternativa_overlay_8_2_3/ as the kit root. Do not stack it onto the global profile without isolating the environment: the existing delegates can force their engine and Mesa again. Migration first requires a controlled rollback of the profile, authorized by the VM owner. This documentation update did not perform that migration.

PR #1400 proposes numerical corrections to gz-rendering; it is not a Parallels-profile installer. Native Jenkins builds passed on commit 9ed780e for Linux, macOS and Windows with zero reported failures. Linux executed the new tests; macOS explicitly skips them and Windows normally disables them. Three GitHub Actions workflows still require maintainer approval. The PR is now ready for review, with code owner iche033 requested by GitHub.

Local PR validation contains 30 process runs across four runtime variants, two new GTest functions and 12 scan configurations. The complete GPU-ray executable gave 11 passes and one expected skip on each of the two fully tested backends. Local source used rendering8 8.3.0, while the public PR targets main with another ABI. Do not copy main libraries into Jazzy or assume this PR fixes the Mesa 25 black noisy-camera failure or all OGRE 2 limitations.

Use 04_SPIEGAZIONE_ORALE_PATCH_IT_EN.txt and bilingual report 03 to understand and discuss the patch. PR sources and evidence are in gazebo_documentation_v3.zip under documentation_v3/pr/. Do not automatically apply them over the global kit's sources/gazebo-virgl.patch: they have different bases and purposes.

19. WHAT TO RECORD ON THE CLEAN VM

Before this procedure, record macOS, Parallels and Tools versions, the Ubuntu image checksum, virtual CPU/RAM, display resolution/scaling, session type, ROS/Gazebo packages and glxinfo. Record each installation step, including STOP conditions and sudo commands. This investigation has not yet completed a second clean VM.

After installation, verify independently: renderer and loaded libraries; observable GUI flicker; the GUI camera FPS counter; actual robot-camera images; /scan and analytical distances; the fan origin at the sensor; an ordinary gz sim command and an ordinary ros2 launch. Keep results even if they differ from the historical measurements. Then run a longer session with motion and representative load: short smoke tests do not certify hours of stability.

For a fair comparison, report resolution, world, sensors and requested rates beside performance. The historical approximately 1.5 mm result, the approximately 31 mm residual in the narrow ablation fixture and the 5 mm tolerance of the new GTest describe three different conditions. They are not one universal accuracy specification.

This revision updates and organizes documents. It did not modify the VM, reinstall its profile, change models or produce new FPS measurements. Runtime scripts remain those retained in the original guide; the later evidence clarifies their context.

SOURCES, KIT AND TESTING LIMITATIONS

[1] Official ROS 2 Jazzy documentation: Ubuntu 24.04 and the ROS environment. Official source consulted in the ros2_documentation repository:

https://github.com/ros2/ros2_documentation/blob/jazzy/source/Installation/Ubuntu-Install-Debs.rst

[2] Parallels: VirGL/Virtio GPU for Linux VMs and guest-side verification:

https://kb.parallels.com/en/128518

[3] Gazebo: the Jazzy/Harmonic pairing and installing ros_gz from the ROS repository:

https://gazebosim.org/docs/harmonic/ros_installation/

[4] The gz-rendering source version pinned by the kit and the upstream LiDAR orientation correction:

https://github.com/gazebosim/gz-rendering/tree/gz-rendering8_8.2.3

https://github.com/gazebosim/gz-rendering/pull/1378

The other results, including the Mesa 24/25 differences, single-row issue, distances, rates and GUI recovery, come from direct tests on the case-study VM. They are not general performance claims for every Mac.

The kit was rebuilt in a temporary directory without reusing the previously modified diagnostic source files; compilation, assembly and operation of the rebuilt runtime were verified. A second Ubuntu VM was not reinstalled from scratch: the generic installer was reviewed and includes version/hash checks and diversion management.

The archive includes a text copy of this guide, source files and scripts. It does not contain prebuilt Mesa packages: prepare.sh downloads them from the configured Ubuntu repositories. The Gazebo source SHA256 check is pinned in the kit.

Purpose: make the verified workaround reproducible, with checks that prevent a smooth window or an existing topic from being mistaken for a correctly functioning sensor.